```
#PID<0.112.0>

iex(6)> spawn fn -> 1 + 2 end
#PID<0.114.0>
```

You will notice that the unnamed function is spawned as various processes that are shown with their PIDs (in *iex(4)*, *iex(5)* and *iex(6)*). Primarily, *spawn* takes a function and executes it in another process.

You can check the status of the process with the following code:

```
iex(7)> pid = spawn fn -> 1 + 2 end
#PID<0.119.0>

iex(8)> Process.alive?(pid)
false

iex(9)>
```

Messages can be sent and received among these processes as shown below:

```
iex> send self(), {:hello, "world"}
{:hello, "world"}
```

Elixir input and output

In Elixir, the IO module is used to perform the basic input and output operations. It can use the standard input and output, files or other IO devices.

```
iex(10)> IO.puts("OSFY")
OSFY
:ok
iex(11)> IO.gets("yes or no? ")
yes or no? yes
"yes\n"

iex(12)>
```

```
The basic file IO can be performed by
using the following code:

iex(12)> {:ok, file} = File.
open("hello", [:write])
{:ok, #PID<0.134.0>}

iex(13)> IO.binwrite(file, "OSFY")
:ok

iex(14)> File.close(file)
:ok

iex(15)>  File.read("hello")
{:ok, "OSFY"}
```

Elixir libraries

One of the major advantages of Elixir is its interoperability with Erlang. Some of the popular libraries are shown in Table 1.

Apart from this, the Erlang ecosystem has various packages that are available through the package manager *https://hex.pm/*.

A simple digraph example with the shortest path is shown in the following code snippet:

```
 digraph = :digraph.new()
{:digraph, #Reference <0.2880715758.
4100325377.247127>,
 #Reference <0.2880715758.4100325377.
247128>,
 #Reference <0.2880715758.4100325377.
247129>, true}

iex(17)> coords = [{0.0, 0.0}, {1.0,
0.0}, {1.0, 1.0}]
[{0.0, 0.0}, {1.0, 0.0}, {1.0, 1.0}]

iex(18)> [v0, v1, v2] = (for
c <- coords, do: :digraph.add_
vertex(digraph, c))
[{0.0, 0.0}, {1.0, 0.0}, {1.0, 1.0}]

iex(19)> :digraph.add_edge(digraph, v0,
v1)
[:"$e" | 0]

iex(20)> :digraph.add_edge(digraph, v1,
v2)
[:"$e" | 1]

iex(21)> :digraph.get_short_
path(digraph, v0, v2)
[{0.0, 0.0}, {1.0, 0.0}, {1.0, 1.0}]
```

To summarise, this article has provided a simple introduction to the Elixir programming language, which is very extensible as it supports the meta-programming concepts. The official documentation has plenty of resources with which you can learn various advanced features such as MIX and OTP. Readers can explore further through various videos, interactive resources, screen casts and books to learn Elixir by navigating to *https://elixir-lang.org/learning.html*. END

Module	Description
binary	This is for handling binary data.
crypto	This module has hashing functions, digital signatures, encryption, etc.
digraph	This is for managing the directed graphs. Facilities for finding the shortest path, etc, are provided.
term storage	This handles storage of large data structures.
math	This handles various mathematical operations.
zip and zlib	This is to perform file zipping.

Table 1: Libraries

References

[1] *https://elixir-lang.org/*
[2] *https://elixir-lang.org/docs.html*

By: Dr K.S. Kuppusamy

The author is an assistant professor of computer science at the School of Engineering and Technology, Pondicherry Central University. He has 13+ years of teaching and research experience in academia and industry. His research interests include accessible computing, Web information retrieval, mobile computing, etc.